Infraestrutura de software
Implementação 5

Atenção

- **Este relatório não é uma entrega separada: ele deve ser enviado junto com o .tar da atividade, no mesma atividade no Google Classroom.**
- **O relatório deve ter como nome as iniciais do seu e-mail em minúsculas (ex.: mla@cesar.school → diretório mla). Fora desse padrão, o relatório é descartado automaticamente.**
- **Cópia identificada, de outro(a) aluno(a) ou de qualquer outra fonte, descarta toda a submissão, código e relatório juntos.**

1. Objetivo (3–5 linhas)

O enunciado pede a sincronização de uma estação de metrô em C com threads POSIX. Um carro chega com assentos livres (station_load_car), passageiros esperam (station_wait_for_car) e avisam que embarcaram (station_on_board). Tudo precisa usar só um mutex e variáveis de condição, sem espera ocupada. Entrego sync_car.c e sync_car.h com station_init e essas três funções, um Makefile que gera o executável sync_car e o teste fornecido passando.

2. Checklist de Requisitos

Um requisito por linha. Sem parágrafo, se precisar explicar algo, use a coluna "como testar".

Requisito do enunciado	Status	Como testar / evidência
station_init inicializa a struct station	OK	make test; o runner chama station_init na linha 69
station_load_car só retorna com o vagão preenchido (todos sentados e carro cheio, ou ninguém esperando)	OK	make test: sem “returned early” nem “Many passengers on this car”
station_load_car retorna na hora sem passageiros esperando ou com 0 assentos	OK	alarmes de 1 s e 2 s do runner não disparam
station_wait_for_car só retorna com carro na estação e assento livre	OK	runner confere que embarcam min(esperando, assentos) em cada carro
Embarque simultâneo: vários passageiros retornam antes de qualquer station_on_board	OK	station_wait_for_car não espera station_on_board (sync_car.c:26-36)
Só pthread_mutex_* e pthread_cond_* permitidas, sem semáforos	OK	grep -n "pthread_|sem_" sync_car.c
No máximo um lock por struct station	OK	sync_car.h tem um único pthread_mutex_t
Sem busy-wait	OK	toda espera é pthread_cond_wait dentro de while (sync_car.c:19-21 e 30-31)
Compilação por Makefile gerando sync_car	OK	make clean && make
struct station declarada em sync_car.c	Parcial	está em sync_car.h, porque o runner só inclui sync_car.h e cria struct station na pilha
Executável em Linux, Unix ou macOS	Parcial	testado só em Ubuntu 26.04 (WSL2); não testei em macOS

3. Como Reproduzir

Comandos reais, copiáveis. Se não rodar com o que está aqui, não é avaliado.

Compilar:

```
make clean
make
```

Executar (exemplo):

```
./sync_car
```

4. Arquitetura (resumida)

Uma linha por arquivo, só o que é específico do seu projeto.

Arquivos e responsabilidades:

1. **sync_car.h → define struct station (um mutex, duas variáveis de condição, três contadores) e os protótipos.**
2. **sync_car.c → implementa as quatro funções.**
3. **sync_car-runner.c → teste do professor com a main, sem alterações.**
4. **Makefile → compila e tem os alvos test, stress e clean.**
5. **evidencias.log → gravação do terminal com a compilação e os testes.**

Decisões técnicas:

1. **Quem embarca reserva o assento (free_seats--) ao sair da espera → dois passageiros não ocupam o mesmo assento e ninguém espera os outros → embarque simultâneo.**
2. **Contador boarding de quem sentou e ainda não chamou station_on_board → o carro não pode partir antes do último aviso → station_on_board acorda o carro só quando boarding chega a 0.**
3. **broadcast em car_arrived e signal em passengers_seated → vários passageiros esperam o carro, mas só o carro espera os passageiros → com signal em car_arrived só um passageiro acordaria e os outros ficariam parados.**
4. **Condição de espera do carro: (waiting > 0 && free_seats > 0) || boarding > 0 → o carro só parte cheio ou sem ninguém esperando, e com todos sentados → retorna na hora com 0 assentos ou sem passageiros.**

5. Estratégias e Diário de Desenvolvimento

Isto é o núcleo do relatório.

5.1 Estratégias da semana

Uma linha por abordagem diferente que você tentou. Se você não mudou de abordagem, preencha só S1.

Estratégia	Nome curto	Contexto	Motivo da troca (se houve)
S1	Reserva de assento + contador de embarque	Um só mutex, duas variáveis de condição (car_arrived, passengers_seated) e os contadores waiting, free_seats, boarding. Primeiro um esqueleto com funções vazias para ver o teste falhar, depois a implementação	não houve troca
S2			
S3			

5.2 Diário de Tentativas

Uma linha por tentativa relevante, marcando a qual estratégia (S1/S2/S3) ela pertence, obviamente precisa ser em ordem cronológica. A coluna "Quando" precisa bater com a ordem cronológica do evidencias.log, é isso que comprova que o fluxo é real. "Hipótese/Causa" só é preenchida quando o resultado foi falha ou parcial.

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
1	S1	Rodar o teste com o esqueleto (só station_init, as outras três funções vazias)	Falhou (esperado)	Funções vazias: station_load_car retorna na hora, sem esperar ninguém embarcar;	19:30:19	

#	Estrat.	O que tentei	Resultado	Hipótese/Causa (se falhou)	Quando	Evidência
				o runner acusa returned early		
2	S1	Implementar station_load_car , station_wait_for _car e station_on_boar d e rodar o teste	OK: Looks good!		19:30:30	
3	S1	100 execuções seguidas do runner	OK, sem falha (27,6 s)		19:31:57	
4	S1	100 execuções presas em uma CPU (taskset -c 0)	OK, sem falha (30,1 s)		19:32:35	
5	S1	8 laços × 60 execuções em paralelo	OK, sem falha (17,4 s)		19:33:05	
6	S1	Compilar com ThreadSanitizer e rodar	Parcial: 5 avisos, 0 citando station_	Os avisos não envolvem a estação; a origem está na linha 9	19:33:37	
7	S1	make stress STRESS_RUN S=20	OK		19:36:50	
8	S1	Extrair o .tar em pasta limpa, compilar e rodar	OK		19:41:59	
9	S1	Listar quais pontos o ThreadSanitizer apontou	Parcial: 7 avisos, 0 citando station_	A saída da linha 6 não existia mais no /tmp, então refiz a execução. São 5 data races e 2 “thread leak”, todos em linhas de sync_car-runner.c (88 e 109 = variáveis do carro, 127 e 137 = variáveis globais, 87 e 111 = threads sem join)	20:28:53	

6. Evidências

A regra continua: toda afirmação relevante precisa de prova. O que muda é como capturar essa prova, para não depender de print manual toda hora.

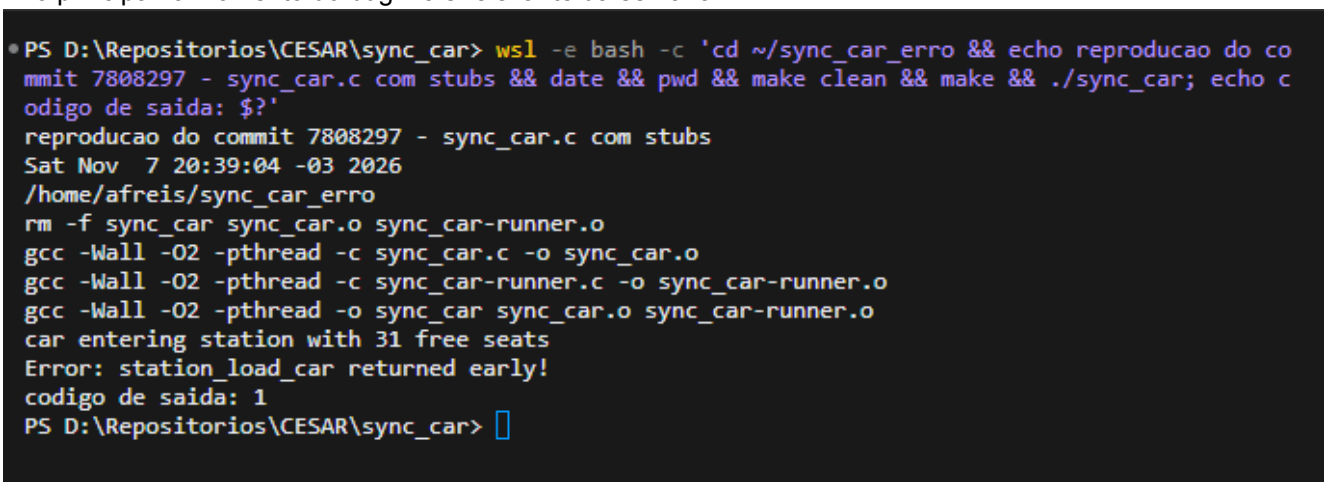
6.1 Log automático (faz o trabalho pesado por você)

No início da sessão de trabalho, rode:

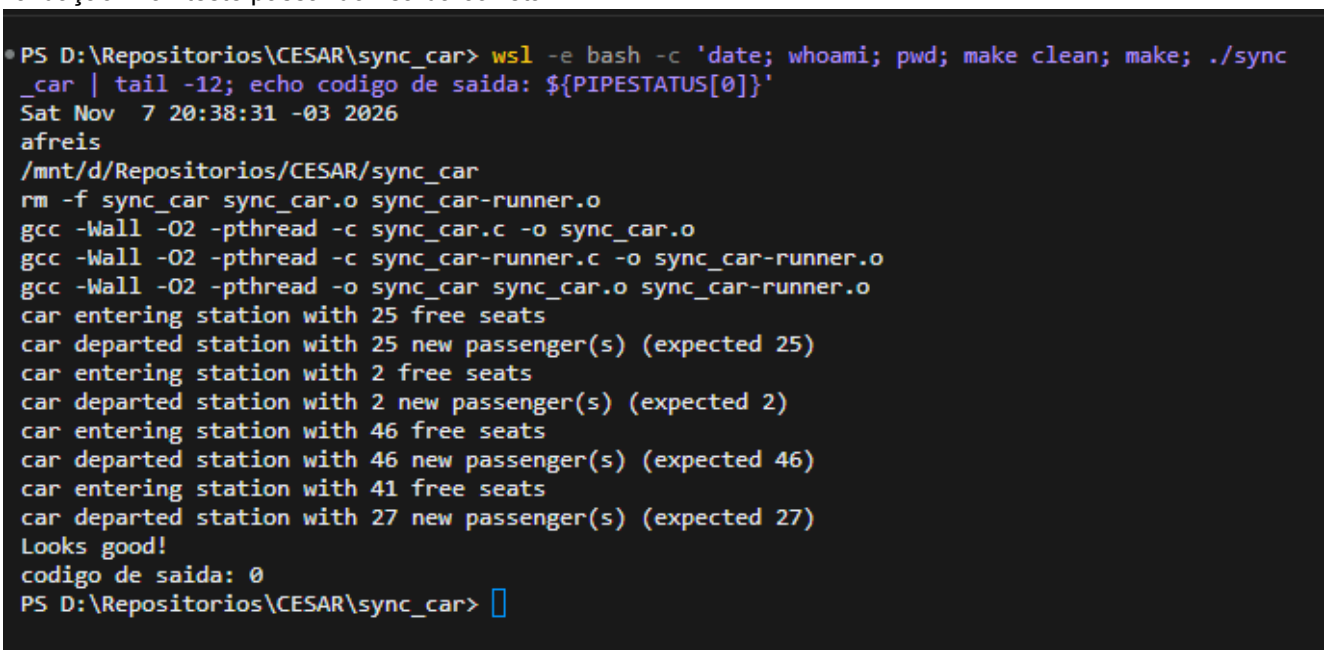
```
script -a evidencias.log
```

6.2 Prints

- Erro principal: o momento do bug mais relevante da semana.



- Validação final: teste passando / saída correta.



7. Uso de IA

Onde usei IA (2–3 linhas): eu usei ia pra ajudar a formatar o relatório e pra me ajudar a planejar como fazer a atividade, o claude me ajudou a rodar os scripts do evidencias pra testar.

O que validei manualmente e como: avaliei os scripts e pra ver se tava funcionando mesmo e fiz o codigo a mao

8. Reflexão Final (5–8 linhas)

A decisão técnica principal foi usar um único mutex com duas variáveis de condição (car_arrived para os passageiros, passengers_seated para o carro) e três contadores (waiting, free_seats, boarding). Quem embarca reserva o assento antes de sair da espera, o que permite embarque simultâneo, e o carro só parte quando ninguém está sentando sem ter avisado com station_on_board. Com mais tempo eu escreveria testes próprios para o que o runner não cobre (carro sem passageiros, passageiro que chega no meio do carregamento, carros em sequência, contagem inválida) e testaria em macOS, que não foi testado. Erro que evitaria: supor que o /tmp do WSL persistia e perder a saída do ThreadSanitizer, o que obrigou a refazer a execução (log, linhas 267–292).

Reflexão:

9. Checklist Final de Entrega

Item	Confirmado?
Compilei do zero seguindo só o que está no relatório	OK
Rodei os testes e evidencias.log foi gerado	OK
Tenho prints obrigatórios	OK
Testei pelo menos um caso limite e um caso inválido	OK
Preenchi a seção de uso de IA	OK
Revisei o relatório e removi frases genéricas / vazias	OK

10. Se eu tivesse mais 2 horas

Pôr -Wextra -Wpedantic -Werror no Makefile (hoje só -Wall; a revisão compilou limpo assim, mas fora do Makefile e do log)

Regras de Integridade (não mudam)

- Ocorrência relevante sem evidência (log ou print) = não conta para a nota.
- Contradição entre prints, log e relato = relatório perde credibilidade.
- Indício de evidência fabricada = entrega invalidada.